Open a Terminal window, navigate to the Catalog Service root folder, and run the newly added tests as follows:

```
$ ./gradlew test --tests BookRepositoryJdbcTests
```

If you experience any failures, it might be because you haven't updated the test cases where the `Book()` constructor is used. We have added new fields to the domain model, so remember to update those test cases as well.

### *12.4.3 Protecting user data with Spring Security and Spring Data R2DBC*

Similar to what we did in Catalog Service, this section will show you how to add data auditing for users in Order Service. Thanks to the abstractions provided by Spring Data and Spring Security, the implementation will not be much different, even though we're using Spring Data R2DBC and reactive Spring.

Besides data auditing, Order Service has one additional critical requirement. Users should be able to access only their own orders. We need to ensure the privacy of all that data. This section will also walk you through the changes necessary to accomplish that result.

## *DEFINING AN AUDITOR TO CAPTURE WHO CREATED OR UPDATED AN R2DBC DATA ENTITY*

Even in this case, we need to tell Spring Data where to get the information about the currently authenticated user. Since it's a reactive application, this